LAB ASSIGNMENT – 2
DATE – 19.01.2026

NAME : Soma Chatterjee
REGISTRATION NO. : AU/2023/0009439

ROLL NO. : UG/02/BTCSECSF/2023/013

SUBJECT : Unstructured Database Lab
SEC: E

Q1. Q1. Make a schema first and then insert 6 documents.

- Roll_no=[1,2,3,4,5,]
- Name=["Ram","Alex","John","Bob","Mukesh",”Danny”]
- Age=[20,19,40,55,30,28]
- Salary=[546.7,333.4,666.7,678.4,245.6,546.3]
- City=["A","B","C","D","E",”F”]
- Phone_no=[123,456,122,444,567,892]

```
mongosh mongodb://127.0.0.
Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Soma Chatterjee>mongosh
Current Mongosh Log ID: 69d2adff8a539f45227c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-04T20:26:52.211+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use companyDB
switched to db companyDB
```

```
companyDB> db.employees.insertMany([
|   { roll_no: 1, name: "Ram", age: 20, salary: 546.7, city: "A", phone_no: 123 },
|   { roll_no: 2, name: "Alex", age: 19, salary: 333.4, city: "B", phone_no: 456 },
|   { roll_no: 3, name: "John", age: 40, salary: 666.7, city: "C", phone_no: 122 },
|   { roll_no: 4, name: "Bob", age: 55, salary: 678.4, city: "D", phone_no: 444 },
|   { roll_no: 5, name: "Mukesh", age: 30, salary: 245.6, city: "E", phone_no: 567 },
|   { roll_no: 6, name: "Danny", age: 28, salary: 546.3, city: "F", phone_no: 892 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d2ae5b8a539f45227c2907'),
    '1': ObjectId('69d2ae5b8a539f45227c2908'),
    '2': ObjectId('69d2ae5b8a539f45227c2909'),
    '3': ObjectId('69d2ae5b8a539f45227c290a'),
    '4': ObjectId('69d2ae5b8a539f45227c290b'),
    '5': ObjectId('69d2ae5b8a539f45227c290c')
  }
}
```

Q2. Write a query to update the name of RAM to SAM.

```
companyDB> db.employees.updateOne(
|    { name: "Ram" },
|    { $set: { name: "Sam" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Q3. Write a query to display only the cities present in that collection.

```
companyDB> db.employees.find({}, { _id: 0, city: 1 })
[
  { city: 'A' },
  { city: 'B' },
  { city: 'C' },
  { city: 'D' },
  { city: 'E' },
  { city: 'F' }
]
```

Q4. Write a query to update the salary by 10%.

```
companyDB> db.employees.updateMany(
|    {},
|    { $mul: { salary: 1.1 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 6,
  upsertedCount: 0
}
```

Q5. Write a query to display all the documents in ascending and descending order of age.

```
companyDB> db.employees.find().sort({ age: 1 })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2908'),
    roll_no: 2,
    name: 'Alex',
    age: 19,
    salary: 366.74,
    city: 'B',
    phone_no: 456
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2907'),
    roll_no: 1,
    name: 'Sam',
    age: 20,
    salary: 601.3700000000001,
    city: 'A',
    phone_no: 123
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290c'),
    roll_no: 6,
    name: 'Danny',
    age: 28,
    salary: 600.93,
    city: 'F',
    phone_no: 892
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290b'),
    roll_no: 5,
    name: 'Mukesh',
    age: 30,
    salary: 270.16,
    city: 'E',
    phone_no: 567
  },
```

```
  {
    _id: ObjectId('69d2ae5b8a539f45227c2909'),
    roll_no: 3,
    name: 'John',
    age: 40,
    salary: 733.3700000000001,
    city: 'C',
    phone_no: 122
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290a'),
    roll_no: 4,
    name: 'Bob',
    age: 55,
    salary: 746.24,
    city: 'D',
    phone_no: 444
  }
]
```

```
companyDB> db.employees.find().sort({ age: -1 })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c290a'),
    roll_no: 4,
    name: 'Bob',
    age: 55,
    salary: 746.24,
    city: 'D',
    phone_no: 444
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2909'),
    roll_no: 3,
    name: 'John',
    age: 40,
    salary: 733.3700000000001,
    city: 'C',
    phone_no: 122
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290b'),
    roll_no: 5,
    name: 'Mukesh',
    age: 30,
    salary: 270.16,
    city: 'E',
    phone_no: 567
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290c'),
    roll_no: 6,
    name: 'Danny',
    age: 28,
    salary: 600.93,
    city: 'F',
    phone_no: 892
  },
```

```
  {
    _id: ObjectId('69d2ae5b8a539f45227c2907'),
    roll_no: 1,
    name: 'Sam',
    age: 20,
    salary: 601.3700000000001,
    city: 'A',
    phone_no: 123
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2908'),
    roll_no: 2,
    name: 'Alex',
    age: 19,
    salary: 366.74,
    city: 'B',
    phone_no: 456
  }
]
```

Q6. Write a query to display all the documents with City → A,B,C.

```
companyDB> db.employees.find({ city: { $in: ["A", "B", "C"] } })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2907'),
    roll_no: 1,
    name: 'Sam',
    age: 20,
    salary: 601.3700000000001,
    city: 'A',
    phone_no: 123
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2908'),
    roll_no: 2,
    name: 'Alex',
    age: 19,
    salary: 366.74,
    city: 'B',
    phone_no: 456
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2909'),
    roll_no: 3,
    name: 'John',
    age: 40,
    salary: 733.3700000000001,
    city: 'C',
    phone_no: 122
  }
]
```

Q7. Write a query to display only two documents from the entire collection.

```
companyDB> db.employees.find().limit(2)
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2907'),
    roll_no: 1,
    name: 'Sam',
    age: 20,
    salary: 601.3700000000001,
    city: 'A',
    phone_no: 123
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2908'),
    roll_no: 2,
    name: 'Alex',
    age: 19,
    salary: 366.74,
    city: 'B',
    phone_no: 456
  }
]
```

Q8. Write a query to delete a document with ROLL_NO:5.

```
companyDB> db.employees.deleteOne({ roll_no: 5 })
{ acknowledged: true, deletedCount: 1 }
```

Q9. Write a query to display all the documents with AGE greater than 20.

```
companyDB> db.employees.find({ age: { $gt: 20 } })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2909'),
    roll_no: 3,
    name: 'John',
    age: 40,
    salary: 733.3700000000001,
    city: 'C',
    phone_no: 122
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290a'),
    roll_no: 4,
    name: 'Bob',
    age: 55,
    salary: 746.24,
    city: 'D',
    phone_no: 444
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290c'),
    roll_no: 6,
    name: 'Danny',
    age: 28,
    salary: 600.93,
    city: 'F',
    phone_no: 892
  }
]
```

Q10. Write a query to display all the documents with AGE less than 20.

```
companyDB> db.employees.find({ age: { $lt: 20 } })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2908'),
    roll_no: 2,
    name: 'Alex',
    age: 19,
    salary: 366.74,
    city: 'B',
    phone_no: 456
  }
]
companyDB> db.employees.find({ age: 20 })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2907'),
    roll_no: 1,
    name: 'Sam',
    age: 20,
    salary: 601.3700000000001,
    city: 'A',
    phone_no: 123
  }
]
```

Q11. Write a query to display all the documents with AGE equals to 20.

```
companyDB> db.employees.find({ age: 20 })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2907'),
    roll_no: 1,
    name: 'Sam',
    age: 20,
    salary: 601.3700000000001,
    city: 'A',
    phone_no: 123
  }
]
```

Q12. Write a query to display all the documents with AGE not equals to 20.

```
companyDB> db.employees.find({ age: { $ne: 20 } })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2908'),
    roll_no: 2,
    name: 'Alex',
    age: 19,
    salary: 366.74,
    city: 'B',
    phone_no: 456
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c2909'),
    roll_no: 3,
    name: 'John',
    age: 40,
    salary: 733.3700000000001,
    city: 'C',
    phone_no: 122
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290a'),
    roll_no: 4,
    name: 'Bob',
    age: 55,
    salary: 746.24,
    city: 'D',
    phone_no: 444
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290c'),
    roll_no: 6,
    name: 'Danny',
    age: 28,
    salary: 600.93,
    city: 'F',
    phone_no: 892
  }
]
```

Q13. Write a query to display all the documents where AGE is greater than equals to 30

```
companyDB> db.employees.find({ age: { $gte: 30 } })
[
  {
    _id: ObjectId('69d2ae5b8a539f45227c2909'),
    roll_no: 3,
    name: 'John',
    age: 40,
    salary: 733.3700000000001,
    city: 'C',
    phone_no: 122
  },
  {
    _id: ObjectId('69d2ae5b8a539f45227c290a'),
    roll_no: 4,
    name: 'Bob',
    age: 55,
    salary: 746.24,
    city: 'D',
    phone_no: 444
  }
]
companyDB> |
```